

can have a number of attributes. One such attribute is the direction, which decides the direction in which the cursor should move. Another attribute is the location, which decides where in the 2D plane the cursor should be placed. There are other attributes also associated with the turtle, like colour, line width, etc. Basically, a Logo program will trace the steps taken by the cursor to generate a graphical image. Even though this seems easy, even simple Logo programs can generate complex images and this will be really interesting to young children who start learning programming.

Over the years, there have been different implementations of the Logo programming language. UCBLLogo, also known as Berkeley Logo, is a very popular implementation of this programming language. Figure 1 shows the logo of UCBLLogo.

However, the graphical user interface of Logo is minimal and not very attractive. Anything intended for children should be visually appealing, so I will discuss KTurtle, which is another free and open source implementation of Logo. KTurtle is an educational programming environment for the KDE Desktop Environment licensed under the GNU General Public License. A KDE utility may not interest GNOME users, but it is possible to install and run KDE applications in GNOME. KTurtle is only loosely based on Logo, unlike UCBLLogo, which is a strict implementation of Logo. Figure 2 shows the logo of KTurtle.

Installing and using KTurtle

Installation of KTurtle is very simple. I have installed it in both Fedora and Ubuntu. In Fedora, you can simply type the name 'KTurtle' in the Software Center, and the installation will be done directly. The same goes for Ubuntu. The Ubuntu Software Center allows you to directly install KTurtle. In both the cases, you should have administrator privileges. I am sure the installation will be similar and as simple in other flavours of Linux also. Once the installation is over, execute the command 'kturtle' on the terminal to open the KTurtle environment. Figure 3 shows the KTurtle programming environment.

If you look at the extreme left part of Figure 3, you will see the editor, which is used to type and execute KTurtle programs. This editor is very powerful with features like intuitive syntax highlighting, line numbering, error markers, visual execution, etc. To the right of the editor you will see the canvas with a green turtle placed in the middle. From now on, all we are going to do is to trace and mark the path traversed by this green turtle with the help of KTurtle programs. Since this is a short tutorial on KTurtle we will only discuss the editor and the canvas. But remember, the KTurtle environment has a few hacks for those who are patient enough to explore further.

To make things easier, KTurtle itself provides some examples with which we can start learning. Let us explore a simple example provided called 'triangle'. You can load



Figure 1: The logo of UCBLLogo

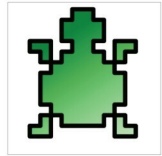


Figure 2: The logo of KTurtle

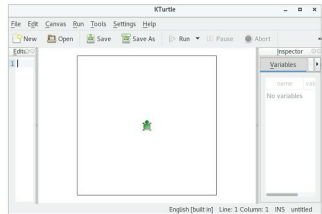


Figure 3: KTurtle programming environment

this program from the menu by selecting the menu item *File > Examples > triangle*. A slightly modified version of the KTurtle program 'triangle' is shown below.

```
reset
cansvsize 500, 500
repeat 3 {
    forward 100
    turnleft 120
}
```

The extra line added to the program 'triangle' is 'cansvsize 500, 500' which sets the canvas width and height to 500 pixels. Now click on the button called 'Run' on the menu bar to view the output on the canvas. The image of a triangle is displayed on the screen. Now add the following lines of code at the end of the program 'triangle' to make some changes to the image drawn on the canvas.

```
go 400, 400
repeat 4 {
    forward 100
    turnleft 90
}
```

Press the button 'Run' again to view the output of this modified 'triangle' program. Figure 4 shows this output,